

1. Dataset target is created in the database, along with the tables using following csv files.

- **customers.csv**
- **sellers.csv**
- **order_items.csv**
- **geolocation.csv**
- **payments.csv**
- **reviews.csv**
- **orders.csv**
- **products.csv**

Problem Statement:

Assuming you are a data analyst/ scientist at Target, you have been assigned the task of analysing the given dataset to extract valuable insights and provide actionable recommendations.

What does 'good' look like?

- 1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset:**
 - 1. Data type of all columns in the "customers" table.**

Following is the Customers table with its columns.

 customers

 QUERY

OPEN IN 

SCHEMA

DETAILS

PREVIEW

TABL

 Filter Enter property name or value

☐	Field name	Type
☐	customer_id	STRING
☐	customer_unique_id	STRING
☐	customer_zip_code_prefix	INTEGER
☐	customer_city	STRING
☐	customer_state	STRING

i. The time range between which orders were placed

Untitled query
▶ RUN
📄 SAVE ▼
⬇️ DOWNLOAD
+ 👤

```

1 select min(order_purchase_timestamp) start_date,
2 max(order_purchase_timestamp) end_date
3 from `target.orders`;

```

Query results

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTIO
Row	start_date ▼	end_date ▼			
1	2016-09-04 21:15:19 UTC	2018-10-17 17:30:18 UTC			

ii. Count the Cities & States of customers who ordered during the given period

Untitled query
▶ RUN
📄 SAVE ▼
⬇️ DOWNLOAD
+ 👤 SHARE ▼
🕒 SCHEDULE
🔗 OPEN IN ▼

```

1 select
2   count(distinct customer_city) no_of_cities,
3   count(distinct customer_state) no_of_states
4 from `target.customers` c join `target.orders` o
5 on c.customer_id = o.customer_id
6 where o.order_purchase_timestamp between "2016-09-04 21:15:19" and "2018-10-17 17:30:18";
7

```

Query results

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DETAILS	EXECUTION GRAPH
Row	no_of_cities ▼	no_of_states ▼				
1	4119	27				

2. In-depth Exploration:

1. Is there a growing trend in the no. of orders placed over the past years?

```
1 select extract(year FROM order_purchase_timestamp) year,  
2 count(order_id) no_of_orders_yearly  
3 from `target.orders`  
4 group by year  
5 order by year;
```

Query results

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION D
Row	year ▼	no_of_orders_yearly			
1	2016	329			
2	2017	45101			
3	2018	54011			

Since concrete results can't be deduced from yearly information,

Following query is run to get more information monthly as well

<
Product ... ory
orders
*Untitled que

Untitled query
RUN
SAVE

```

1 select extract(year FROM order_purchase_timestamp) year,
2 extract(month FROM order_purchase_timestamp) month,
3 count(order_id) no_of_orders
4 from `target.orders`
5 group by year,month
6 order by year,month;

```

Query results

[SAVE RESULT](#)

	JOB INFORMATION	RESULTS	CHART	JSON
Row	year	month	no_of_orders	
1	2016	9	4	
2	2016	10	324	
3	2016	12	1	
4	2017	1	800	
5	2017	2	1780	
6	2017	3	2682	
7	2017	4	2404	
8	2017	5	3700	
9	2017	6	3245	
10	2017	7	4026	
11	2017	8	4331	
12	2017	9	4285	
13	2017	10	4631	
14	2017	11	7544	
15	2017	12	5673	
16	2018	1	7269	
17	2018	2	6728	
18	2018	3	7211	

[Load more](#)

- Can we see some kind of monthly seasonality in terms of the no. of orders being placed?

2017 February onwards orders picked up.

Product ... ory X orders X *Untitled que

Untitled query RUN SAVE

```
1 select extract(year FROM order_purchase_timestamp) year,
2 extract(month FROM order_purchase_timestamp) month,
3 count(order_id) no_of_orders
4 from `target.orders`
5 group by year,month
6 order by year,month;
```

Query results SAVE RESULT

	JOB INFORMATION	RESULTS	CHART	JSON
Row	year	month	no_of_orders	
1	2016	9	4	
2	2016	10	324	
3	2016	12	1	
4	2017	1	800	
5	2017	2	1780	
6	2017	3	2682	
7	2017	4	2404	
8	2017	5	3700	
9	2017	6	3245	
10	2017	7	4026	
11	2017	8	4331	
12	2017	9	4285	
13	2017	10	4631	
14	2017	11	7544	
15	2017	12	5673	
16	2018	1	7269	
17	2018	2	6728	
18	2018	3	7211	

Load more

If we do Monthly analysis 5th, 7th and 8th months have max orders

Untitled query

RUNSAVE

```
1 select
2 extract(month FROM order_purchase_timestamp)
3 count(order_id) no_of_orders
4 from `target.orders`
5 group by month
6 order by month;
```

Query results

JOB INFORMATION		RESULTS	CHART
Row	month	no_of_orders	
1	1	8069	
2	2	8508	
3	3	9893	
4	4	9343	
5	5	10573	
6	6	9412	
7	7	10318	
8	8	10843	
9	9	4305	
10	10	4959	
11	11	7544	
12	12	5674	

3. During what time of the day, do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)
- 0-6 hrs : Dawn
 - 7-12 hrs : Mornings
 - 13-18 hrs : Afternoon
 - 19-23 hrs : Night

Untitled query
RUN
SAVE
DOWNLOAD
SHARE
SCHEDULE
OPEN IN

```

1 SELECT
2     time_of_the_day,
3     COUNT(order_id) AS orders
4 FROM (
5     SELECT
6         order_id,
7         CASE
8             WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6 THEN 'Dawn'
9             WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12 THEN 'Morning'
10            WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18 THEN 'Afternoon'
11            ELSE 'Night'
12        END AS time_of_the_day
13     FROM `target.orders`
14 ) t
15 GROUP BY time_of_the_day
16 ORDER BY orders DESC;

```

Query results

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DETAILS	EXECUTION GRAPH
Row	time_of_the_day	orders				
1	Afternoon	38135				
2	Night	28331				
3	Morning	27733				
4	Dawn	5242				

As we can see during afternoon, orders are more when compared to other time of the day, and dawn has least orders.

3. Evolution of E-commerce orders in the Brazil region:

1. Get the month-on-month no. of orders placed in each state.


```
1 select EXTRACT(year FROM o.order_purchase_timestamp) year,
2         EXTRACT(month FROM o.order_purchase_timestamp) month,
3         c.customer_state,
4         count(o.order_id) orders_placed
5 from `target.orders` o left join `target.customers` c
6 on o.customer_id = c.customer_id
7 group by year,month,customer_state
8 order by year,month;
```

Query results

SAVE RESULTS

OPEN IN

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DETAILS
Row	year	month	customer_state	orders_placed	
1	2016	9	SP	2	
2	2016	9	RS	1	
3	2016	9	RR	1	
4	2016	10	CE	8	
5	2016	10	SP	113	
6	2016	10	RJ	56	
7	2016	10	MT	3	
8	2016	10	GO	9	
9	2016	10	SC	11	
10	2016	10	RS	24	
11	2016	10	MG	40	
12	2016	10	PR	19	
13	2016	10	DF	6	
14	2016	10	SE	3	
15	2016	10	ES	4	
16	2016	10	BA	4	
17	2016	10	PE	7	
18	2016	10	RN	4	
19	2016	10	PA	4	
20	2016	10	MA	4	
21	2016	10	AL	2	
22	2016	10	PB	1	
23	2016	10	PI	1	
24	2016	10	RR	1	
25	2016	12	PR	1	
26	2017	1	SP	299	
27	2017	1	PR	65	
28	2017	1	ES	12	
29	2017	1	RJ	97	
30	2017	1	MG	108	
31	2017	1	SC	31	
32	2017	1	GO	18	
33	2017	1	RN	5	
34	2017	1	RS	54	
35	2017	1	DF	13	
36	2017	1	MS	1	
37	2017	1	BA	25	
38	2017	1	CE	9	
39	2017	1	SE	4	
40	2017	1	MT	11	
41	2017	1	PE	9	
42	2017	1	PI	7	
43	2017	1	RO	3	
44	2017	1	AC	2	
45	2017	1	RR	2	

2. How are the customers distributed across all the states?

Following is the report of customers distributed state wise in ascending order.

```
1 select customer_state,  
2     count(customer_unique_id) no_of_customers  
3 from `target.customers`  
4 group by customer_state  
5 order by no_of_customers asc ;
```

Query results

JOB INFORMATION		RESULTS	CHART	JSON
row	customer_state ▼	no_of_customers ▼		
1	RR	46		
2	AP	68		
3	AC	81		
4	AM	148		
5	RO	253		
6	TO	280		
7	SE	350		
8	AL	413		
9	RN	485		
10	PI	495		
11	PB	536		
12	MS	715		

Following is the report of customers distributed state wise in descending order.

```
Untitled query [RUN] [SAVE] [DOWNLOAD]
1 select customer_state,
2     count(customer_unique_id) no_of_customers
3 from `target.customers`
4 group by customer_state
5 order by no_of_customers desc ;
```

Query results

JOB INFORMATION		RESULTS	CHART	JSON
Row	customer_state	no_of_customers		
1	SP	41746		
2	RJ	12852		
3	MG	11635		
4	RS	5466		
5	PR	5045		
6	SC	3637		
7	BA	3380		
8	DF	2140		
9	ES	2033		
10	GO	2020		
11	PE	1652		
12	CE	1336		

4. **Impact on Economy: Analyse the money movement by e-commerce by looking at order prices, freight and others.**

1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).

You can use the "payment value" column in the payments table to get the cost of orders.

```

1 WITH cte AS (
2     SELECT
3         EXTRACT(YEAR FROM order_purchase_timestamp) AS year,
4         SUM(payment_value) AS cost_of_order
5     FROM `target.payments` p
6     JOIN `target.orders` o ON p.order_id = o.order_id
7     WHERE
8         EXTRACT(YEAR FROM order_purchase_timestamp) IN (2017, 2018)
9         AND EXTRACT(MONTH FROM order_purchase_timestamp) BETWEEN 1 AND 8
10    GROUP BY year
11 )
12 SELECT
13     ROUND (MAX(CASE WHEN year = 2017 THEN cost_of_order END),2) AS cost_of_2017,
14     ROUND (MAX(CASE WHEN year = 2018 THEN cost_of_order END),2) AS cost_of_2018,
15     ROUND(
16         ((MAX(CASE WHEN year = 2018 THEN cost_of_order END) -
17          MAX(CASE WHEN year = 2017 THEN cost_of_order END)) /
18          MAX(CASE WHEN year = 2017 THEN cost_of_order END)) * 100, 2
19     ) AS percentage_increase
20 FROM cte;

```

Query results

JOB INFORMATION	RESULTS	CHART	JSON	EXECUTION DETAILS	EXE
w	cost_of_2017 ▼	cost_of_2018 ▼	percentage_increase		
1	3669022.12	8694733.84	136.98		

- Calculate the Total & Average value of order price for each state.

```

1 SELECT
2 c.customer_state,
3 ROUND(SUM(p.payment_value), 2) AS total_value,
4 ROUND(AVG(p.payment_value), 2) AS avg_value
5 FROM `target.customers` c
6 JOIN `target.orders` o ON c.customer_id = o.customer_id
7 JOIN `target.payments` p ON o.order_id = p.order_id
8 GROUP BY c.customer_state
9 ORDER BY total_value desc;

```

Query results

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION
row	customer_state ▼	total_value ▼	avg_value ▼		
1	SP	5998226.96	137.5		
2	RJ	2144379.69	158.53		
3	MG	1872257.26	154.71		
4	RS	890898.54	157.18		
5	PR	811156.38	154.15		
6	SC	623086.43	165.98		
7	BA	616645.82	170.82		
8	DF	355141.08	161.13		
9	GO	350092.31	165.76		
10	ES	325967.55	154.71		

- Calculate the Total & Average value of order freight for each state.

```

1 SELECT
2 c.customer_state,
3 ROUND(SUM(oi.freight_value), 2) AS total_freight,
4 ROUND(AVG(oi.freight_value), 2) AS average_freight
5 FROM `target.order_items` oi
6 JOIN `target.orders` o ON oi.order_id = o.order_id
7 JOIN `target.customers` c ON o.customer_id = c.customer_id
8 GROUP BY c.customer_state
9 ORDER BY total_freight DESC;

```

Query results

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION
row	customer_state ▼	total_freight ▼	average_freight ▼		
1	SP	718723.07	15.15		
2	RJ	305589.31	20.96		
3	MG	270853.46	20.63		
4	RS	135522.74	21.74		
5	PR	117851.68	20.53		
6	BA	100156.68	26.36		
7	SC	89660.26	21.47		
8	PE	59449.66	32.92		

5. Analysis based on sales, freight and delivery time.

- Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

- time_to_deliver** = order_delivered_customer_date - order_purchase_timestamp
- diff_estimated_delivery** = order_delivered_customer_date - order_estimated_delivery_date

Untitled query
▶ RUN
📄 SAVE
⬇️ DOWNLOAD
👤 SHARE
🕒 SCHEDULE
🔗 OPEN IN
⚙️ MORE

```

1 select order_id,
2 order_purchase_timestamp,
3 order_estimated_delivery_date,
4 order_delivered_customer_date,
5 date_diff(order_delivered_customer_date, order_purchase_timestamp, DAY) actual_delivery_time,
6 date_diff(order_delivered_customer_date, order_estimated_delivery_date, DAY) estimated_delivery_time
7 from `target.orders`
8 ORDER BY order_id;

```

Query results

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DETAILS		EXECUTION GRAPH	
Row	order_id	order_purchase_timestamp	order_estimated_delivery_date	order_delivered_customer_date	actual_delivery_time	estimated_delivery_time		
1	00010242fe8c5a6d1ba2dd792...	2017-09-13 08:59:02 UTC	2017-09-29 00:00:00 UTC	2017-09-20 23:43:48 UTC	7	-8		
2	00018f77f2f0320c557190d7a1...	2017-04-26 10:53:06 UTC	2017-05-15 00:00:00 UTC	2017-05-12 16:04:24 UTC	16	-2		
3	000229ec398224ef6ca0657da...	2018-01-14 14:33:31 UTC	2018-02-05 00:00:00 UTC	2018-01-22 13:19:16 UTC	7	-13		
4	00024acbcdf0a6daa1e931b03...	2018-08-08 10:00:35 UTC	2018-08-20 00:00:00 UTC	2018-08-14 13:32:39 UTC	6	-5		
5	00042b26cf59d7ce69dfabb4e...	2017-02-04 13:57:51 UTC	2017-03-17 00:00:00 UTC	2017-03-01 16:42:31 UTC	25	-15		
6	00048cc3ae777c65dbb7d2a06...	2017-05-15 21:42:34 UTC	2017-06-06 00:00:00 UTC	2017-05-22 13:44:35 UTC	6	-14		
7	00054e8431b9d7675808bcb8...	2017-12-10 11:53:48 UTC	2018-01-04 00:00:00 UTC	2017-12-18 22:03:38 UTC	8	-16		
8	000576fe39319847cbb9d288c...	2018-07-04 12:08:27 UTC	2018-07-25 00:00:00 UTC	2018-07-09 14:04:07 UTC	5	-15		
9	0005a1a1728c9d785b8e2b08...	2018-03-19 18:40:33 UTC	2018-03-29 00:00:00 UTC	2018-03-29 18:17:31 UTC	9	0		
10	0005f50442cb953dcd1d21e1f...	2018-07-02 13:59:39 UTC	2018-07-23 00:00:00 UTC	2018-07-04 17:28:31 UTC	2	-18		

2. Find out the top 5 states with the highest & lowest average freight value.

```

1 SELECT customer_state, average_freight_value
2 FROM (
3   (SELECT
4     c.customer_state,
5     ROUND(AVG(oi.freight_value), 2) AS average_freight_value
6   FROM `target.order_items` oi
7   JOIN `target.orders` o ON oi.order_id = o.order_id
8   JOIN `target.customers` c ON o.customer_id = c.customer_id
9   GROUP BY c.customer_state
10  ORDER BY average_freight_value DESC
11  LIMIT 5)
12 UNION ALL
13 (SELECT
14   c.customer_state,
15   ROUND(AVG(oi.freight_value), 2) AS average_freight_value
16 FROM `target.order_items` oi
17 JOIN `target.orders` o ON oi.order_id = o.order_id
18 JOIN `target.customers` c ON o.customer_id = c.customer_id
19 GROUP BY c.customer_state
20 ORDER BY average_freight_value ASC
21 LIMIT 5
22 )) AS highest_lowest
23 ;

```

Query results

[SAVE RESULTS](#)

	JOB INFORMATION	RESULTS	CHART	JSON
row	customer_state ▾	average_freight_value		
1	RR	42.98		
2	PB	42.72		
3	RO	41.07		
4	AC	40.07		
5	PI	39.15		
6	SP	15.15		
7	PR	20.53		
8	MG	20.63		
9	RJ	20.96		
10	DF	21.04		

- Find out the top 5 states with the highest & lowest average delivery time.

Untitled query

RUN

DOWNLOAD

SAVE

Query

```

1 SELECT customer_state, average_delivery_time
2 FROM (
3   (SELECT
4     c.customer_state,
5     ROUND(AVG(DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY)), 2) AS
6     average_delivery_time
7   FROM `target.orders` o
8   JOIN `target.customers` c ON o.customer_id = c.customer_id
9   WHERE order_delivered_customer_date IS NOT NULL
10  GROUP BY c.customer_state
11  ORDER BY average_delivery_time DESC
12  LIMIT 5)
13  UNION ALL
14  (SELECT
15    c.customer_state,
16    ROUND(AVG(DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY)), 2) AS
17    average_delivery_time
18  FROM `target.orders` o
19  JOIN `target.customers` c ON o.customer_id = c.customer_id
20  WHERE order_delivered_customer_date IS NOT NULL
21  GROUP BY c.customer_state
22  ORDER BY average_delivery_time ASC
23  LIMIT 5)
24 ) AS highest_lowest;

```

Press Alt+F1 for Accessibility

Query results

SAVE RESULTS
 OPEN IN

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DETAILS	EXECUTION
w	customer_state	average_delivery_time				
1	RR	28.98				
2	AP	26.73				
3	AM	25.99				
4	AL	24.04				
5	PA	23.32				
6	SP	8.3				
7	PR	11.53				
8	MG	11.54				
9	DF	12.51				
10	SC	14.48				

- Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.
You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

Untitled query
 ▶ RUN
📄 SAVE ▾
⬇️ DOWNLOAD
👤 SHARE ▾
🕒 SCHEDULE
🔗 OPEN IN ▾

```

1 SELECT
2   c.customer_state,
3   ROUND(AVG(DATE_DIFF(order_delivered_customer_date, order_purchase_timestamp, DAY)), 2) AS actual,
4   ROUND(AVG(DATE_DIFF(order_estimated_delivery_date, order_purchase_timestamp, DAY)), 2) AS estimated
5 FROM `target.orders` o
6 JOIN `target.customers` c ON o.customer_id = c.customer_id
7 GROUP BY c.customer_state
8 HAVING actual < estimated
9 ORDER BY actual
10 LIMIT 5;
    
```

Query results

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DETAILS	EXECUTION GRAPH
Row	customer_state ▾	actual ▾	estimated ▾			
1	SP	8.3	18.81			
2	PR	11.53	24.25			
3	MG	11.54	24.22			
4	DF	12.51	24.06			
5	SC	14.48	25.42			

6. Analysis based on the payments:

- Find the month-on-month no. of orders placed using different payment types.

```

1 select
2 extract(year FROM o.order_purchase_timestamp) as year,
3   extract(month FROM o.order_purchase_timestamp) as month,
4   p.payment_type,
5   count(distinct o.order_id) orders_placed
6 from `target.orders` o join `target.payments` p
7 on o.order_id = p.order_id
8 group by year,month,p.payment_type
9 order by year asc, month desc,p.payment_type;

```

Press Alt+F1 f

Query results

[SAVE RESULTS](#)  

JOB INFORMATION		RESULTS	CHART	JSON	EXECUTION DETAILS	E
Row	year ▼	month ▼	payment_type ▼	orders_placed ▼		
1	2016	12	credit_card	1		
2	2016	10	UPI	63		
3	2016	10	credit_card	253		
4	2016	10	debit_card	2		
5	2016	10	voucher	11		
6	2016	9	credit_card	3		
7	2017	12	UPI	1160		
8	2017	12	credit_card	4363		
9	2017	12	debit_card	64		
10	2017	12	voucher	220		
11	2017	11	UPI	1509		
12	2017	11	credit_card	5867		
13	2017	11	debit_card	70		
14	2017	11	voucher	267		
15	2017	10	UPI	993		

- Find the no. of orders placed on the basis of the payment instalments that have been paid.

🔍 **Untitled query** ▶ RUN + 👤 SHARE ▾

```
1
2 select payment_installments,
3 count(distinct order_id) as no_of_orders
4 from `target.payments`
5 where payment_value > 0
6 group by payment_installments
7 order by payment_installments;
```

Query results

JOB INFORMATION		RESULTS	CHART
Row	payment_installment	no_of_orders ▾	
1	0	2	
2	1	49057	
3	2	12389	
4	3	10443	
5	4	7088	
6	5	5234	
7	6	3916	
8	7	1623	
9	8	4253	
10	9	644	
11	10	5315	
12	11	23	
13	12	133	
14	13	16	
15	14	15	
16	15	74	